

Python Pattern Questions

Beginner to Intermediate level Python pattern questions with:

- Difficulty tags
- Pattern output
- Approach explanation
- Python code

These questions help improve nested loops, logic building, and problem-solving skills.

Follow **RahulXTech** on Instagram for more coding content:

@rahulxtech

1. Square Star Pattern [\[Easy\]](#)

Pattern Output:

```
* * * *
* * * *
* * * *
* * * *
```

Approach: Use fixed rows and columns.

Python Code:

```
n = 4

for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()
```

2. Right Triangle [\[Easy\]](#)

Pattern Output:

```
*
**
***
****
```

Approach: Stars depend on row number.

Python Code:

```
n = 4

for i in range(1, n + 1):
    for j in range(i):
        print("*", end="")
    print()
```

3. Reverse Triangle [\[Easy\]](#)

Pattern Output:

```
****
***
**
*
```

Approach: Decrease stars every row.

Python Code:

```
n = 4

for i in range(n, 0, -1):
    for j in range(i):
        print("*", end="")
    print()
```

4. Pyramid Pattern [\[Medium\]](#)

Pattern Output:

```
  *
 ***
*****
*****
```

Approach: Use spaces and odd-number stars.

Python Code:

```
n = 4

for i in range(1, n + 1):
    for j in range(n - i):
        print(" ", end="")

    for j in range(2 * i - 1):
        print("*", end="")

    print()
```

5. Inverted Pyramid [\[Medium\]](#)

Pattern Output:

```
*****
*****
***
*
```

Approach: Increase spaces and decrease stars.

Python Code:

```
n = 4

for i in range(n):
    for j in range(i):
        print(" ", end="")

    for j in range(2 * (n - i) - 1):
        print("*", end="")

    print()
```

6. Diamond Pattern [\[Medium\]](#)

Pattern Output:

```
  *
 ***
*****
 ***
  *
```

Approach: Combine pyramid and inverted pyramid.

Python Code:

```
n = 3

for i in range(1, n + 1):
    for j in range(n - i):
        print(" ", end="")

    for j in range(2 * i - 1):
        print("*", end="")

    print()

for i in range(n - 1, 0, -1):
    for j in range(n - i):
        print(" ", end="")

    for j in range(2 * i - 1):
        print("*", end="")

    print()
```

7. Hollow Square [\[Medium\]](#)

Pattern Output:

```
****
*  *
*  *
****
```

Approach: Print stars only on borders.

Python Code:

```
n = 4

for i in range(n):
    for j in range(n):
        if i == 0 or i == n-1 or j == 0 or j == n-1:
            print("*", end="")
        else:
            print(" ", end="")

    print()
```

8. Binary Pattern [\[Medium\]](#)

Pattern Output:

```
1
01
101
0101
```

Approach: Use modulo logic.

Python Code:

```
n = 4

for i in range(n):
    for j in range(i + 1):
        if (i + j) % 2 == 0:
            print(1, end="")
        else:
            print(0, end="")

    print()
```

9. Number Triangle [\[Easy\]](#)

Pattern Output:

```
1
12
123
1234
```

Approach: Print numbers till current row.

Python Code:

```
n = 4

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end="")

    print()
```

10. Floyd's Triangle [\[Medium\]](#)

Pattern Output:

```
1
23
456
78910
```

Approach: Use global counter variable.

Python Code:

```
n = 4
num = 1

for i in range(1, n + 1):
    for j in range(i):
```

```
        print(num, end="")
        num += 1

    print()
```

11. Square Star Pattern [\[Easy\]](#)

Pattern Output:

```
* * * *
* * * *
* * * *
* * * *
```

Approach: Use fixed rows and columns.

Python Code:

```
n = 4

for i in range(n):
    for j in range(n):
        print("*", end=" ")
    print()
```

12. Right Triangle [\[Easy\]](#)

Pattern Output:

```
*
**
***
****
```

Approach: Stars depend on row number.

Python Code:

```
n = 4

for i in range(1, n + 1):
    for j in range(i):
        print("*", end="")
    print()
```

13. Reverse Triangle [\[Easy\]](#)

Pattern Output:

```
****
***
**
*
```

Approach: Decrease stars every row.

Python Code:

```
n = 4

for i in range(n, 0, -1):
    for j in range(i):
```

```

      *
    * * *
  * * * * *
    * * *
      *

```

Approach: Combine pyramid and inverted pyramid.

Python Code:

```
n = 3

for i in range(1, n + 1):
    for j in range(n - i):
        print(" ", end="")

    for j in range(2 * i - 1):
        print("*", end="")

    print()

for i in range(n - 1, 0, -1):
    for j in range(n - i):
        print(" ", end="")

    for j in range(2 * i - 1):
        print("*", end="")

    print()
```

17. Hollow Square [\[Medium\]](#)

Pattern Output:

```
****
*  *
*  *
****
```

Approach: Print stars only on borders.

Python Code:

```
n = 4

for i in range(n):
    for j in range(n):
        if i == 0 or i == n-1 or j == 0 or j == n-1:
            print("*", end="")
        else:
            print(" ", end="")

    print()
```

18. Binary Pattern [\[Medium\]](#)

Pattern Output:

```
1
01
101
0101
```

Approach: Use modulo logic.

Python Code:

```
n = 4

for i in range(n):
```

```
for j in range(i + 1):
    if (i + j) % 2 == 0:
        print(1, end="")
    else:
        print(0, end="")
print()
```

19. Number Triangle [\[Easy\]](#)

Pattern Output:

```
1
12
123
1234
```

Approach: Print numbers till current row.

Python Code:

```
n = 4

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end="")
    print()
```

20. Floyd's Triangle [\[Medium\]](#)

Pattern Output:

```
1
23
456
78910
```

Approach: Use global counter variable.

Python Code:

```
n = 4
num = 1

for i in range(1, n + 1):
    for j in range(i):
        print(num, end="")
        num += 1
    print()
```

Keep practicing consistently.

Follow RahulXTech on Instagram: [@rahulxtech](#)